```
aspace = 3
1
tot_params = 128*100 + 25*100 + aspace*25*51
2
theta = torch.randn(tot_params)/10.
3
theta.requires_grad=True
theta_2 = theta.detach().clone()
4

vmin,vmax= -10,10
gamma=0.9
lr = 0.00001
update_rate = 75
5
support = torch.linspace(-10,10,51)
state = torch.randn(2,128)/10.
6
action_batch = torch.Tensor([0,2])
7
reward_batch = torch.Tensor([0,10])
8
losses = []
pred_batch = dist_dqn(state,theta,aspace=aspace)
9
target_dist = get_target_dist(pred_batch,action_batch,reward_batch,
\
                                    support, lim=
(vmin,vmax),gamma=gamma)  10

plt.plot((target_dist.flatten(start_dim=1)
[0].data.numpy()),color='red',label='target')
plt.plot((pred_batch.flatten(start_dim=1)
[0].data.numpy()),color='green',label='pred')
plt.legend()
```

- ***1*** Defines the action space to be of size 3
- ***2*** Defines the total number of Dist-DQN parameters based on layer sizes
- ***3*** Randomly initializes a parameter vector for Dist-DQN
- ***4*** Clones theta to use as a target network
- ***5*** Synchronizes the main and target Dist-DQN parameters every 75 steps
- ***6*** Randomly initializes two states for testing
- ***7*** Creates synthetic action data
- ***8*** Creates synthetic reward data